

Alexandre Plana

@alexandre-plana

Full-stack developer with front-end expertise

Industrial applications · UI/UX · Cross-platform

About

I develop industrial applications, drawing on a strong front-end and UI/UX background that I now extend across the full stack. On Xistar, I work from the operator interface through to the services that ingest and distribute operational data. My approach connects user needs, architecture and technical constraints, with close attention to product consistency from the initial design to delivered code.

R&D Consultant · Datacorp / X-PERT

France

alexandre.plana.pro@gmail.com

+33 6 71 03 97 84

Languages

French Native · English Technical

Core Skills

LANGUAGES

typescript · go · c# · python

FRONT-END & VISUALIZATION

react · vue 3 · single-spa · dockview · maplibre · deck.gl · unity 3d

SERVICES & INTEROPERABILITY

api rest · mqtt · ais · nmea · cot · mavlink

QUALITY & TOOLS

jest · vitest · sonarqube · gitlab ci · docker · git · postman · jira · figma

Interests

🎮 gaming · 🚵 mountain biking · 🥾 hiking · 🍳 cooking

Online resume
alexandre-plana.github.io/resume/

📌 Career overview

Datacorp — 2026 — R&D Consultant – X-PERT

x-pert/xistar — Full-stack developer

Xistar – Maritime Tactical Command Center · 2026

web · multi-screen · real-time · react · typescript · go · docker · gitlab ci

x-pert/ai-workflow — AI workflow design & development

AI agent execution workflow · 2026

ai · bmad · docker · gitlab ci · react

x-pert/xpert-design-system — UI design & front-end architecture

X-PERT design system – from Figma to applications · 2026

web · ux/ui · figma · react · shadcn · npm

Schneider Electric — 2017 - 2026 — R&D Consultant – Datacorp

schneider-electric/iiot-edge-ieca — Front-end developer

Industrial Automation Edge (IECA) · 2023 - 2026

web · vue 3 · typescript · single-spa · pinia · jest · sonarqube

schneider-electric/aoa-ar-app — C# / Unity Developer

Augmented Operator Advisor (AOA) – Industrial AR Application · 2017 - 2023

3d / xr · unity · c# · ar · mobile · offline

Oreka Engineering — 2013 - 2017 — C# / Unity 3D Developer

oreka/dosicase — UX/UI design

Dosicase – Radiation-safety training simulator · 2015 - 2017

ux/ui · radiation safety · 3d simulation · unity · c# · figma

suez/serious-game-radioprotection — C# developer / Training simulation

Radiation protection – Nuclear safety training · 2014 - 2015

serious game · radiation safety · nuclear · unity · c#

simulhom — Functional design & UX/UI

Musculoskeletal disorder prevention tool · 2015 - 2017

ux/ui · ergonomics · 3d simulation · unity · c#

chu-caen/emsim-basic — C# developer / 3D medical simulation

Radiation protection for interventional surgery · 2013 - 2015

3d simulation · radiation safety · medical · unity · c#

areva/emem — C# developer / 3D simulation

EMEM mobile equipment evacuation enclosure – Nuclear safety · 2014 - 2016

3d simulation · nuclear · unity · c# · wpf

edf/aeroball — C# developer / Serious game

Aeroball – Training application for nuclear control-station operators · 2014 - 2015

serious game · nuclear · unity · c#

suez/maintenance-generateurs-de-vapeurs — C# developer / Serious game

Steam-generator maintenance – Training for nuclear outages · 2014 - 2015

serious game · nuclear · unity · c#

📌 Vue.js · micro-frontend – Schneider Electric

Schneider Electric 2017 – 2026
R&D Consultant – Datacorp

📁 **schneider-electric/iiot-edge-ieca** — Front-end developer

Industrial Automation Edge (IECA) · 2023 – 2026

Within a multidisciplinary #scrum team, I developed Edge applications organized as #microfrontend modules with #vue3, #typescript and #single-spa. My work focused primarily on interfaces and front-end architecture, with occasional back-end feature extensions. I contributed to the platform's functional areas, shared components, tests and code-quality monitoring.

KEY ACTIONS

- Contribute to features for connectors, computations, conditions, alerts and scripting.
- Build shared components in #vue3 and #typescript, including a configurable table, a routing foundation and a property selector.
- Extend back-end features where required by the front-end work.
- Write unit tests for components, #pinia stores and utility functions with #jest, then monitor performance and code quality with #sonarqube.
- Design an IP Sprint prototype for a #mobile application that sends alerts to on-call operators outside the IECA platform.

vue3 · ts · single-spa · docker · jest · html · css · git · figma · jira

IECA MICRO-FRONTENDS I WORKED ON

I worked on each of the micro-frontends below, all built with #vue3 and #typescript and loaded as #single-spa applications, with #pinia, Vue Router and Schneider Electric's Quartz design system. The newer applications share one architecture: container/presentational components, typed DTO-to-model mappers, one store per domain and forms validated with vee-validate and yup or zod.

SHARED FOUNDATION	
🧩 **@ieca/ieca-web-utils** Shared library	MainTable, PropertiesPicker, Stepper, RoutableHeader with breadcrumbs, import and confirmation dialogs, drop zone; common stores (dialogs, notifications, rights, application registry); HTTP client with interceptors; typings and test-utility packages. vue3 · typescript · tailwind · axios · npm
🧩 **ieca-vue3-ui-template** Micro-frontend template	Starting point for new applications: router filtered by write permissions, dialog, notification and rights stores, #single-spa and #jest setup. vue3 · single-spa · jest
DATA & CONNECTIVITY	
🧩 **ieca-connectivity-ui** OT/IT connectivity	The largest application (96 components): OPC UA and Modbus connectors created, duplicated and configured through generated forms, variables edited in an ag-grid with dynamic cells, CSV import, actions, and interfaces → groups → properties on the IT side. vue3 · ag-grid · monaco · yup · tailwind
🧩 **ieca-data-management-ui** Data management	Filterable data-model tree (filter kept in session), properties and their assignments (variable, computation, source), adding properties and levels, import, exportable history and value writes. vue3 · monaco · yup · tailwind
🧩 **ieca-datamodel-ui** Data model	Tree editor for the data model: creating and deleting levels, mapping properties through PropertiesPicker, import. A rewrite of the former assets-and-devices application for the new API. vue3 · pinia · tailwind

PROCESSING & MONITORING	
🧩 `ieca-computation-ui` **Computations**	Computed variables started or stopped from the list; step-by-step wizard: properties, formula validated server-side with a function catalog, then a sliding or tumbling time window, with previews. `vue3` `typescript` `pinia`
🧩 `ieca-condition-ui` **Conditions**	If/Then rules built step by step: properties and quality level, formula validated server-side, then an action (application, command or script); JSON Schema transformation for parameters. `vue3` `typescript` `json schema`
🧩 `ieca-scripting-ui` **Scripting**	Scripts created through a generated form and edited in Monaco with suggestions, enabled, run or cancelled; executions and parsed logs; navigation guards for rights and editing. `vue3` `monaco` `cron` `yup`
🧩 `ieca-alert-manager-ui` **Alerts**	Alert list and activity history, acknowledgement and closing, bulk deletion through JSON Patch and CSV export. `vue3` `typescript` `floating-vue`
PLATFORM ADMINISTRATION	
🧩 `ieca-platform-configuration-ui` **Platform configuration**	Server certificate and key, database disk usage, configuration export and import with an error report, application settings through generated forms. `vue3` `monaco` `yup` `tailwind`
🧩 `ieca-mail-ui` **SMTP mail**	SMTP server configuration (host, security, credentials, certificate checked with node-forge) and a test message with connection status. `vue3` `typescript` `node-forge`
🧩 `ieca-overview-ui` **Overview**	“Applications” page: cards for the installed Edge applications with their provider and license. `vue3` `tailwind`
PROTOTYPES	
🧩 `alexandre-plana/ip-sprint-*` **IP Sprint – data on mobile (2024)**	A mobile version of IECA data management, using Capacitor for `#android` and iOS and then Quasar, and an experimental mobile shell able to load `#single-spa` micro-frontends. `vue3` `capacitor` `quasar` `mobile`
🧩 `ip-sprint · notifications` **IP Sprint – on-call notifications**	A prototype that takes IECA alerts beyond the platform: they feed a mobile-oriented stream and become push notifications on a dedicated mobile app, for example for an on-call operator. `mobile` `push notifications`

📌 Web development – X-PERT

Datacorp 2026
R&D Consultant – X-PERT

📁 x-pert/xistar

Full-stack developer

Xistar – Maritime Tactical Command Center · 2026

Xistar brings together tactical awareness, mission and drone control, alert management, multi-screen operation and mission replay. Based on operational requirements, I contribute to the functional design of operator workflows, interactions and expected feature behavior. The interface uses `#typescript`, `#react` and `#dockview`; mapping relies on `#maplibre` and `#deckgl` to display NATO `#app-6d` symbology, an `#mgrs` grid and bathymetry. `#go` services ingest partner feeds through `#ais`, `#nmea`, `#cot` and `#mavlink`, transmit commands and publish information through `#mqtt` and `#api-rest`. Deployment profiles and communication between instances allow the product to adapt to defense, port and demonstration environments.

Multi-screen & real time

Multi-screen application driven from a master station: screen sets, a mission board and remote view sending, with centralized, real-time control of what each screen displays. Situation, alert and position updates stream in through `#mqtt` and `#socketio`, and time-based replay lets operators review an operation for analysis and debriefing.

Containers & pipelines

Interface and `#go` services containerized with `#docker`, with a development workflow and `#gitlab-ci` pipelines that build and ship the images.

🗂️ x-pert/xistar-fleet-sim

Drone & naval fleet simulator · 2026

Developed a drone and naval fleet simulator that supplies Xistar with data streams representative of operating conditions. Built in `#typescript` on `#nodejs`, it generates positions, states and detections and publishes them through `#mqtt` for demonstrations and feature development without relying on static mock data.

🗂️ x-pert/xistar-mobile

Mobile app – alerts & orders · 2026

Developed a mobile application that lets operators receive BAM/CAM alerts and issue commands from the field. Built in `#typescript` with `#react`, it supports alert escalation and return orders and communicates with Xistar through `#socketio`.

🗂️ x-pert/xistar-wall-emulator

Screen wall emulator · 2026

Developed a screen-wall emulator for designing and demonstrating Xistar multi-view workflows without an equipped control room. Built in `#typescript` with `#react` and `#vite`, it reproduces screen groups, a mission board and remote view dispatch from the master workstation.

KEY ACTIONS

- Contribute to Xistar's functional design based on operational requirements: define operator workflows, interactions and expected feature behavior.
- Build the `#typescript` and `#react` operator workspace around `#dockview` panels, then integrate `#maplibre` and `#deckgl` mapping with NATO `#app-6d` symbology.
- Develop the alert system and its escalation mechanisms through a dedicated `#api` and email notifications.
- Implement interoperability in the `#go` services: ingest `#ais`, `#nmea`, `#cot` and `#mavlink` partner feeds, transmit commands and publish data through `#mqtt` and `#api-rest`.
- Develop the multi-screen application and centralized control from a master workstation, then implement mission replay for analysis, situation reports and debriefings.

`typescript` `react` `go` `dockview` `maplibre` `deck.gl` `mqtt` `api rest` `docker` `gitlab ci`

📁 x-pert/xpert-design-system

UI design & front-end architecture

X-PERT design system – from Figma to applications · 2026

X-PERT's design system covers technical applications, the company website and the visual styling of official documents and videos. Components are designed in `#figma`, developed in `#typescript` with `#react` and `#shadcn`, then published to a private `#npm` registry. The library combines standard elements—cards, buttons, switches and tables—with technical interface components: charts and treemaps, activity feeds, timelines, maps, tabs and resizable panels. Internal documentation describes their integration into applications and their use in official documents.

KEY ACTIONS

- Design interfaces and components in `#figma` for technical applications and the company website.
- Build the library in `#typescript` and `#react` on top of `#shadcn`: cards, buttons, switches, tables, charts, treemaps, activity feeds, timelines, maps, tabs and resizable panels.
- Publish components to a private `#npm` registry for distribution across company applications.
- Define the visual styling of official documents and videos, and write internal documentation for applications and documents.

`typescript` `react` `shadcn` `figma` `npm` `design system`

📁 x-pert/ai-workflow

AI workflow design & development

AI agent execution workflow · 2026

I designed an AI-agent-assisted development workflow using the `#bmad` method as its foundation. My personal project Iasit also helped me become familiar with agent coordination, execution tracking and improvement loops, and that experience informed the project's design. Each project has its own `#docker` image for running agents in an isolated environment, while runs are dispatched to a dedicated machine through `#gitlab-ci`. A tool built with `#react` and `#typescript` uses agent hooks to track progress as execution unfolds. Session reports feed a self-improvement loop for both the agents and the workflow.

KEY ACTIONS

- Structure an AI-agent-assisted development workflow based on `#bmad` principles.
- Isolate agent execution in `#docker` containers built from a project-specific image.
- Use `#gitlab-ci` to trigger runs and dispatch them to a dedicated machine.
- Build a `#react` and `#typescript` monitoring tool that uses agent hooks to show progress as execution unfolds.
- Analyze session reports and automatically feed the lessons identified back into the agents and workflow.

`bmad` `docker` `gitlab ci` `react` `typescript` `ai agents`

📌 Industrial AR application – Schneider Electric

Schneider Electric 2017 – 2026
R&D Consultant – Datacorp

📁 schneider-electric/aoa-ar-app

C# / Unity Developer

Augmented Operator Advisor (AOA) – Industrial AR Application · 2017 – 2023

For six years, I developed the interfaces and front-end architecture of an industrial `#ar` application in `#csharp` and `#unity`. The scope covered `#android`, `#ios` and `#windows` versions, customization for `#oem` clients, `#offline` operation and preparation for each distribution platform. I also contributed to the project server in `#python` and its `#http` communication with the application.

KEY ACTIONS

- Design the front-end architecture and an interface adapted to `#android`, `#ios` and `#windows` in portrait and landscape modes using `#csharp` and `#unity`.
- Develop `#offline` operation and `#http` synchronization, then contribute to the `#python` project server and network-error handling.
- Maintain backward compatibility by reading legacy project formats and migrating them during loading.
- Implement an externalized style system for producing custom-branded versions for `#oem` clients.
- Create a unit-testing tool in `#unity` integrated into the stages before and after builds.

`unity` `c#` `ar` `mobile` `uiux` `figma` `agile` `jira`

📌 Serious games & simulation – Oreka Engineering

Oreka Engineering `2013 - 2017`

C# / Unity 3D Developer

📁 oreka/xr-projects

C# / Unity 3D Developer

Industrial XR Projects – EDF · RTE · AREVA · DCNS · SUEZ · 2013 – 2017

For four years, I contributed to the functional design and development of interactive applications for industry, nuclear operations and defense. Starting from client requirements and operational procedures, I helped define workflows, scenarios, interactions and functional mockups for `#seriousgame` training applications and simulators. I then implemented them in `#csharp` with `#unity`, integrated models from `#solidworks` and `#3dsmax`, optimized CPU/GPU performance and prepared releases.

KEY ACTIONS

- Contribute to functional design by analyzing client requirements, assessing technical feasibility and formalizing workflows, interactions and mockups.
- Design `#seriousgame` training applications and simulators from operational procedures, then structure scenarios using state graphs, conditions and triggers.
- Develop 2D/3D interfaces and interactive systems in `#csharp` with `#unity`, including interactions for `#vr` and `#ar` devices.
- Integrate and optimize `#cad` models from `#solidworks` and `#3dsmax`: meshes, levels of detail, lightmaps, materials and shaders.
- Optimize CPU/GPU performance with `#unity` profiling tools, then prepare builds, fix defects and support client demonstrations.

`unity` `c#` `wpf` `winform` `solidworks` `3dsmax` `vr` `uiux` `figma` `desktop`

KEY PROJECTS

📁 oreka/dosicase

UX/UI design

Dosicase – Radiation-safety training simulator · 2015 – 2017

Dosicase uses simulation to support practical radiation-safety training. I designed the experience and `#uiux` interface to present situations and radiological risks clearly to learners, then contributed to their integration into the simulator in `#csharp` with `#unity`. The work covered both information architecture and the interactions required for hands-on training.

KEY ACTIONS

- Structure the learning journey from radiation-safety training objectives.
- Design the `#uiux` interface and organize information about situations and radiological risks.
- Define the simulator interactions required for hands-on training.
- Contribute to integrating the experience into the application in `#csharp` with `#unity`.

`uiux` `design` `web` `radioprotection` `dosimetry` `figma`

📁 suez/serious-game-radioprotection

C# developer / Training simulation

Radiation protection – Nuclear safety training · 2014 – 2015

The module used simulated accident scenarios to teach radiation-protection and nuclear-safety procedures. I structured the training sequences, modeled the scenario flow and implemented emergency-team management. I then developed and integrated these mechanisms in `#csharp` with `#unity` in a 3D environment.

KEY ACTIONS

- Translate nuclear-safety procedures into interactive training sequences.
- Design simulated accident scenarios and their progression.
- Implement emergency-team management within the simulation.
- Develop the training module and its 3D environment in `#csharp` with `#unity`.

`unity` `c#` `3d-temps-réel` `training` `education`

📁 simulhom

Functional design & UX/UI

Musculoskeletal disorder prevention tool · 2015 – 2017

Simulhom is a 3D ergonomics simulator designed to analyze working postures and visualize musculoskeletal risks. I contributed to the product's functional design with ergonomists, then designed the workflows and `#uiux` interface to make postural indicators clear and actionable. I also helped develop the application in `#csharp` with `#unity` and integrate visual feedback into the 3D scene.

KEY ACTIONS

- Contribute to functional design with ergonomists by formalizing use cases and analysis scenarios.
- Design workflows, the `#uiux` interface and visual feedback associated with postural risks.
- Develop interactions and 3D simulator components in `#csharp` with `#unity`.
- Integrate postural analysis and present risk indicators throughout the simulation.

`unity` `c#` `3d-temps-réel` `vr` `ergonomics`

OTHER PROJECTS

🗂️ chu-caen/emsim-basic

Radiation protection for interventional surgery · 2013 – 2015

Developed a 3D medical simulation for training interventional-surgery teams in radiation-protection practices. I contributed to representing dosimetry throughout the scenarios, visualizing radiation-exposed areas and integrating operator protocols in `#csharp` with `#unity`.

🗂️ areva/emem

EMEM mobile equipment evacuation enclosure – Nuclear safety · 2014 – 2016

Developed an interactive 3D simulation of EMEM, a mobile enclosure for evacuating equipment in a nuclear environment. I contributed to representing flows and radioactive materials and designed operator control interfaces using `#csharp`, `#unity` and `#wpf`.

🗂️ edf/aeroball

Aeroball – Training application for nuclear control-station operators · 2014 – 2015

Developed a `#seriousgame` to train operators on the Aeroball control station in a nuclear environment. I helped translate operational procedures into interactive scenarios, then implemented them in `#csharp` with `#unity` and modeled the 3D training environment.

🗂️ suez/maintenance-generateurs-de-vapeurs

Steam-generator maintenance – Training for nuclear outages · 2014 – 2015

Developed a `#seriousgame` to prepare personnel for steam-generator maintenance during nuclear outages. I helped translate maintenance procedures into interactive scenarios, then implemented them in `#csharp` with `#unity` in a 3D environment.

🛠️ Personal projects

🛠️ AI-ASSISTED DEVELOPMENT WORKFLOW in service

iasit

Iasit is a coordination tool for software development assisted by AI agents. I designed it to structure preparation, execution, verification, correction and acceptance, track bugs as findings, and turn recurring problems into learned rules. Relevant rules are fed back into future prompts to improve subsequent executions. The application combines `#react` and `#typescript` with a `#nodejs` runtime.

🗓 2026 · Product design & full-stack development

`react` `typescript` `nodejs` `zod` `sigma.js` `api rest` `mcp`

🛠️ 3D EDUCATIONAL SANDBOX active

vorrery

Vorrery is a 3D educational sandbox designed for learning through experimentation, observation and hands-on manipulation. Users build their own systems, adjust parameters and immediately see the consequences of their choices. The product combines free exploration, guided experiments and child-friendly journeys in a `#react` `#typescript` interface rendered with `#threejs` and `#react-three-fiber`.

🗓 2026 · Product design & 3D front-end development

`react` `typescript` `three.js` `r3f` `web workers`

🛠️ SELF-HOSTED APPLICATION design phase

kin

Design of a private, self-hosted family digital memory for keeping photos, videos and documents on a home server. The product roadmap covers an Android gallery, local administration and remote mobile access, with particular attention to reliable storage and data ownership.

🗓 2026 · Product design & architecture

`node` `typescript` `android` `api rest` `sqlite`

🎓 Education

📚 TRAINING

React · TypeScript

Datacorp · 2026

Advanced React and TypeScript training focused on modern front-end development practices

📚 TRAINING

TypeScript · Docker · Vue.js

Datacorp · 2023

Professional training in TypeScript, Docker and Vue.js applied to web application development

📚 INTERNSHIP

Polymorph

Final-year internship · 2013

Graphic design · Visual communication

🎓 DEGREE

Bachelor's degree in real-time 3D

3DI Laval · 2012–2013

Unity 3D · Virtual reality · Real-time 3D

🎓 DEGREE

BTS in Visual Communication – Multimedia option

3DI Laval · 2010–2012

Multimedia · Web · Video